

PySHS Graphical User Interface

User Manual

Based on PySHS V3.2.x (Pierre-Marie Gassin et al.)

Table of Contents

PySHS GUI — User Manual

PySHS V3.2.x Graphical User Interface

A desktop interface for the Computational Second Harmonic Scattering codes **HRS** and **SHS**

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

This user manual accompanies the PySHS Tkinter GUI. It describes how to install, launch, and operate the interface, and how each menu and input parameter maps to the underlying C programs documented in the official PySHS user manual.

To report errors or suggest improvements related to the GUI, contact the PySHS authors (e.g. pierre-marie.gassin@enscm.fr) or the maintainer of this interface.

Contents

1. Introduction
2. Requirements and installation
3. Launching the interface
4. Overview of the main window
5. HRS Calculation tab
6. SHS Calculation tab
7. Positions / Orientations generation

8. Results panel and plots
 9. Input file formats
 10. Practical workflow examples
 11. Important notes and troubleshooting
 12. Relation to the command-line PySHS codes
-

1. Introduction

1.1 Purpose of the GUI

PySHS is a computational package for **Second Harmonic Scattering (SHS)** calculations. The core engines are written in **C** (HRS . c, SHS . c). They are normally run from the terminal with interactive prompts and text input files.

The **PySHS GUI** provides a graphical front-end that:

- selects the calculation module (**HRS** or **SHS**);
- sets all numerical parameters through forms;
- edits or loads the hyperpolarizability tensor (β) and, for SHS, the positions/orientations file;
- compiles the C sources if needed;
- runs the calculation and streams progress messages live;
- displays intensity plots and (for SHS) a 3D dipole-assembly plot;
- allows export of results and figures.

The scientific meaning of the outputs (coefficients (a_V), (b_V), (c_V), (I_2), (I_4), depolarization ratio, angle-resolved intensities, etc.) is the same as in the official PySHS manual. This document focuses on **how to use the interface**.

1.2 What the GUI can compute

Module	Physical situation	Underlying binary
HRS	Incoherent scattering from uncorrelated molecules in solution (Hyper-Rayleigh Scattering)	HRS
SHS	Coherent scattering from N correlated molecules (supramolecular aggregate)	SHS

Calculation types (keywords), shared by both modules:

Keyword	Description
polarplot_single	Polarization-resolved intensity at one fixed scattering

Keyword	Description
	angle Θ
polarplot_integrate	Polarization-resolved intensity integrated between two scattering angles
angle_scattering	Intensity as a function of scattering angle for fixed polarization combinations

2. Requirements and installation

2.1 Software requirements

- **Python 3.8+** with:
 - tkinter (usually included; on Debian/Ubuntu: `sudo apt install python3-tk`)
 - matplotlib (`pip install matplotlib` or system package `python3-matplotlib`)
 - numpy
- A **C compiler**: gcc or clang
 - Linux: `sudo apt install build-essential`
 - macOS: `xcode-select --install`
 - Windows: MinGW-w64, MSYS2, or equivalent providing gcc
- Optional but recommended on Linux: `stdbuf` (GNU coreutils) for live line-buffered output from the C programs

2.2 Directory layout

Typical layout of the GUI package:

```
PySHS_GUI/
├── pyshs_gui.py           # Main application
├── HRS.c / SHS.c         # C sources
├── HRS / SHS             # Executables (created by compilation)
├── script_sphere_random.py
├── script_spheroid_random.py
├── script_cylinder_random.py
├── graph_polar.py
├── graph_angle.py
├── graph_assembly.py
├── work/                 # Temporary job directories
└── PySHS_GUI_User_Manual.md # This manual
```

2.3 Compilation

The GUI can compile the sources automatically (**Compile HRS.c** / **Compile SHS.c** buttons, or at startup).

Manual compilation (equivalent to the official manual):

```
gcc -o HRS HRS.c -lm
gcc -o SHS SHS.c -lm
```

On Windows, executables are named HRS.exe and SHS.exe.

3. Launching the interface

From a terminal, in the GUI directory:

```
python3 pyshs_gui.py
```

On first launch the interface checks for gcc/c++ and compiles HRS.c and SHS.c if the executables are missing or older than the sources. Status messages appear in the **Calculation results** panel at the bottom.

4. Overview of the main window

The window is split into two vertical regions:

1. **Top — calculation tabs**
 - Tab **HRS Calculation**
 - Tab **SHS Calculation**
2. **Bottom — shared results area**
 - Left: text log and numerical output
 - Right: plots (intensity and, for SHS, dipole assembly)

A horizontal sash between top and bottom can be dragged to give more space to the calculation forms or to the results.

5. HRS Calculation tab

Use this tab for **incoherent** Hyper-Rayleigh Scattering (uncorrelated molecules).

5.1 Calculation type

Dropdown **Calculation type**:

Value	Meaning
polarplot_single	Polarization plot at one scattering angle Θ
polarplot_integrate	Polarization plot with integration from a start

Value	Meaning
<code>angle_scattering</code>	angle to an end angle Angular distribution of intensity between 0° and 180°

The parameter fields below change automatically according to the selected type.

5.2 Parameters

Parameter	Unit	When visible	Description
Scattering angle Θ	degrees	<code>polarplot_single</code>	Laboratory scattering angle. 0° = forward / transmission; 90° = right-angle collection (common experimental geometry).
Start angle	degrees	<code>polarplot_integrate</code>	Lower bound of the scattering-angle integration range.
End angle	degrees	<code>polarplot_integrate</code>	Upper bound of the integration range.
Number of points	integer	<code>polarplot_integrate</code> , <code>angle_scattering</code>	Number of samples in the angular range (for <code>integrate</code> : integration points; for <code>angle_scattering</code> : points from 0° to 180°).

5.3 Hyperpolarizability tensor (β)

Text area for the microscopic hyperpolarizability tensor in **atomic units (a.u.)**.

Required format (same as the official PySHS manual):

```
# comment line (optional but recommended)
xxx 0.0
xyx 0.0
xxz 0.0
xyx 0.0
xyy 0.0
xyz 0.0
xzx 0.0
xzy 0.0
xzz 0.0
yxx 0.0
yxy 0.0
yxz 0.0
yyx 0.0
```

```

yyy 0.0
yyz 0.0
yzx 0.0
zyy 0.0
yzz 0.0
zxx 0.0
zxy 0.0
zxz 0.0
zyx 0.0
zyy 0.0
zyz 0.0
zzx 0.0
zzy 0.0
zzz 1.0

```

- Line 1 is treated as a comment by the C program (skipped).
- Each following line: **label** (e.g. zzz) then **value**.
- All 27 Cartesian components of the rank-3 tensor must be present.

Buttons:

- **Load example** — fills the area with a simple example (only ($\{zzz\} = 1$)).
- **Load file...** — loads a β file from disk.

5.4 Actions

-  **Run HRS calculation** — writes temporary input files, feeds interactive answers to HRS, streams stdout to the results panel, then loads plots.
 - **Compile HRS.c** — forces recompilation of the HRS binary.
-

6. SHS Calculation tab

Use this tab for **coherent** Second Harmonic Scattering from a finite set of correlated molecules (aggregate).

6.1 Calculation type

Same keywords as HRS: polarplot_single, polarplot_integrate, angle_scattering.

6.2 Parameters

Parameter	Unit	Always / conditional	Description
Number of dipoles	integer	Always shown	Number of molecules (N) in the aggregate. Must equal the

Parameter	Unit	Always / conditional	Description
			number of lines in the positions/orientations file. The GUI automatically overwrites this field with the line count of the orientation data when you run a calculation (see §11.1).
Wavelength	nm	Always	Laser wavelength () in nanometres (e.g. 800). Used to build wave vectors ($k = 2\pi/\lambda$) and ($2k$).
Refractive index – real part	dimensionless	Always	Real part of the medium refractive index (n) (e.g. 1.33 for water).
Refractive index – imaginary part	dimensionless	Always	Imaginary part of (n) (absorption). Use 0.0 for a non-absorbing medium.
Scattering angle Θ	degrees	polarplot_single	Same meaning as in HRS (0° transmission, 90° right angle).
Start / End angle	degrees	polarplot_integrate	Integration bounds for the scattering angle.
Number of points	integer	polarplot_integrate, angle_scattering	Number of angular samples.

The left column is scrollable; the buttons **Run SHS calculation** and **Compile SHS.c** stay pinned at the bottom of that column.

6.3 Hyperpolarizability (β) sub-tab

Same format and buttons as in the HRS tab (see §5.3). The tensor is the **molecular** hyperpolarizability in the molecular frame; the C code rotates it into the laboratory frame for each dipole using the Euler angles from the orientation file.

6.4 Positions / Orientations sub-tab

This sub-tab holds the mesoscopic description of the aggregate: for each molecule (j), orientation and position in the mesoscopic frame.

File format (one molecule per line, six columns):

φ' θ' ψ' x' y' z'

Column	Symbol	Unit	Meaning
1	φ'	radian	Euler angle (ZYZ convention)
2	θ'	radian	Euler angle
3	ψ'	radian	Euler angle
4–6	x', y', z'	nm (same length unit as wavelength)	Position of the molecule

The number of non-empty valid lines is the number of dipoles (N).

Buttons:

- **Load file...** — load an existing orientation file; updates **Number of dipoles** automatically.
- **Save as...** — save the current text area to a file.

Below these controls is the **generation** panel (see §7).

6.5 Actions

-  **Run SHS calculation** — runs SHS with the current β , orientation data, and parameters.
- **Compile SHS.c** — recompiles the SHS binary.

7. Positions / Orientations generation

In the SHS tab, under **Positions / Orientations**, the panel **Generate positions / orientations file** builds orientation files without leaving the GUI.

7.1 Script selection

Dropdown **Choose a script:**

Menu entry	Script file	Geometry
Sphere (script_sphere_random.py)	script_sphere_random.py	Dipoles on a sphere, radial orientation +

Menu entry	Script file	Geometry
		optional Gaussian disorder
Spheroid (script_spheroid_random.py)	script_spheroid_random.py	Dipoles on a spheroid (semi-axes (a), (c)) + disorder
Cylinder (script_cylinder_random.py)	script_cylinder_random.py	Dipoles on a cylinder surface grid + disorder
Custom script...	User-selected .py file	Any script that writes the 6-column format

7.2 Parameters per script

Sphere

Field	Meaning
Radius (nm)	Sphere radius
Number of dipoles	(N)
σ disorder (rad)	Standard deviation of angular noise around the radial direction (0 = pure radial)

Spheroid

Field	Meaning
Radius a (nm)	Equatorial semi-axis
Radius c (nm)	Polar semi-axis
Number of dipoles	(N)
σ disorder (rad)	Angular disorder

Cylinder

Field	Meaning
Radius (nm)	Cylinder radius
Length (nm)	Cylinder length
Dipoles per radius	Number of angular samples around the circumference
Dipoles along length	Number of samples along the axis
σ disorder (rad)	Angular disorder

Total (N =) (dipoles per radius) $\times$ (dipoles along length).

Custom script

- **Browse...** — select a Python script.
- **stdin answers (one per line)** — answers to the script's interactive `input ( )` prompts, in order.
- The GUI always passes the output file path as `sys.argv[1]` (same convention as the official PySHS scripts).

7.3 Generate button

 **Generate positions / orientations file** runs the selected script, loads the resulting 6-column data into the text area, sets **Number of dipoles** to the line count, and refreshes the **Dipole assembly** plot.

8. Results panel and plots

8.1 Calculation results (text)

The left bottom panel shows:

- compilation messages;
- live stdout/stderr from the C program (progress percentages, angle values, `av`, `bv`, `cv`, ...);
- the content of the main output file written by HRS/SHS;
- warnings (e.g. mismatch between the dipole field and the orientation file).

Buttons:

- **Save...** — save the full log to a text file.
- **Clear** — clear the panel.
-  **Stop calculation** — kill the running C process (enabled only while a job is running).

There is **no time limit** on calculations. Large aggregates ($(N 10^3)-(10^4)$) may run for minutes to hours.

8.2 Plots

Two sub-tabs:

SHS Intensity

- For `polarplot_single` / `polarplot_integrate`: curves **IV(γ)** and **IH(γ)** versus polarization angle γ (degrees).
- For `angle_scattering`: **IV(0°)**, **IV(90°)**, **IH(0°)**, **IH(90°)** versus scattering angle (with symmetrization about 0° as in the official plotting scripts).

Dipole assembly (mainly SHS)

- 3D quiver plot of molecular positions and orientation directions, built from the orientation data (same role as `graph_assembly.py` in the official package).

Export:

- **Export intensity plot...** — PNG or PDF.
 - **Export assembly plot...** — PNG or PDF.
-

9. Input file formats

9.1 Hyperpolarizability file (`input_beta`)

See §5.3. Values in **atomic units**. First line = comment; then 27 labelled components.

9.2 Orientation / position file (SHS only)

See §6.4. Angles in **radians**, positions in **nm** (consistent with wavelength in nm). One molecule per line:

`phi theta psi x y z`

Example (two molecules):

0.000000	1.570796	0.000000	10.000000	0.000000	0.000000
3.141593	1.570796	0.000000	-10.000000	0.000000	0.000000

10. Practical workflow examples

10.1 HRS polarization plot at 90°

1. Open tab **HRS Calculation**.
2. Set **Calculation type** = `polarplot_single`.
3. Set **Scattering angle Θ** = 90.
4. Load or edit the β tensor.

5. Click  **Run HRS calculation.**
6. Inspect coefficients in the log (av, bv, cv, I2v, I4v, ...) and the intensity plot.

10.2 SHS for a spherical aggregate

1. Open tab **SHS Calculation**.
2. Set **Calculation type** = polarplot_single, $\Theta = 90$.
3. Set **Wavelength** = 800, **n real** = 1.33, **n imag** = 0.0.
4. Load the molecular β tensor.
5. In **Positions / Orientations**, choose **Sphere**, set radius and (N), click **Generate**.
6. Confirm that **Number of dipoles** matches the generated line count.
7. Click  **Run SHS calculation.**
8. Review intensity plot and dipole assembly plot; export if needed.

10.3 Angle-resolved SHS

1. Set **Calculation type** = angle_scattering.
 2. Set **Number of points** (e.g. 20 $\rightarrow$ sampling from -180° to $+180^\circ$ in the C code's convention).
 3. Provide β and orientation data.
 4. Run and inspect the angle-resolved curves.
-

11. Important notes and troubleshooting

11.1 Number of dipoles must match the orientation file

The C program asks for (N) and then reads **exactly (N) lines** from the orientation file.

In coherent SHS, intensities scale approximately as (N^2) . If the GUI field **Number of dipoles** differs from the number of lines in the file, results can be wrong by large factors (e.g. a factor of 400 if (N) differs by 20).

The GUI now forces (N) to the line count of the orientation data when a calculation is started, and logs a warning if the field was inconsistent.

When comparing GUI results with a manual terminal run, use the **same** (N), β file, orientation file, wavelength, refractive index, angle, and keyword.

11.2 Large (N) and memory

SHS.c allocates large arrays on the stack sized by (N). Very large (N) (many thousands) may crash or become extremely slow. Prefer testing with hundreds of dipoles before scaling up.

11.3 Live progress not visible

Progress lines require line-buffered stdout. The GUI:

- compiles the C sources with `setvbuf` for line buffering;
- launches processes with `stdbuf -oL -eL` when available.

Some calculation modes print little until the end; `angle_scattering` is usually the most verbose (one message per angle).

11.4 Compiler not found

Install a C compiler and ensure it is on the PATH. Use **Compile HRS.c** / **Compile SHS.c** and read the log panel.

11.5 Stopping a long job

Use  **Stop calculation** in the results panel. The underlying C process is terminated.

12. Relation to the command-line PySHS codes

The GUI does **not** reimplement the physics. It:

1. writes temporary β (and orientation) files;
2. builds the same command line as in the official manual, e.g.

```
./SHS polarplot_single input_beta.txt input_orient.txt output.txt
```
3. supplies the interactive answers (`scanf`) via stdin:
 - SHS: (N), wavelength (nm), ((n)), ((n)), then angle-related values depending on the keyword;
 - HRS: angle-related values only;
4. reads `out_plot` and the main output file for display.

For the definition of laboratory frames, Euler conventions (ZYZ), intensity formulas, and coefficient definitions ((a_V), (b_V), (c_V), (I_2), (I_4), ...), refer to **Chapters 2–4 of the official PySHS V3.2.x user manual**.

Equivalent terminal commands

HRS

```
./HRS <Keyword> <input_beta> <outputfile>
```

SHS

```
./SHS <Keyword> <input_beta> <input_orientation> <outputfile>
```

Keywords: polarplot_single | polarplot_integrate | angle_scattering.

Acknowledgements

PySHS was developed at Institut Charles Gerhardt Montpellier – ENSCM.
Support: CAMOMILS Research National French Agency, ANR-15-CE21-0002.

Contributors to the scientific code include Dr Pierre-Marie Gassin, Dr Lotfi Boudjema, and Dr Gaelle Gassin. Discussions with Prof. Pierre-François Brevet and the precursor work of Dr Julien Duboisset (HRS_computing) are gratefully acknowledged.

Please cite:

J. Chem. Inf. Model. 2020, 60, 12, 5912–5917

when publishing results obtained with PySHS (command line or GUI).

End of the PySHS GUI User Manual